

Arrays & Hashing

A worked study guide: patterns, derivations, tested code, and a recognition system for interviews

Roadmap

This guide follows the same progression as the source study plan: start from array-only tricks that exploit position and contiguity, then move into hashing tricks that exploit fast lookup, and finish by combining the two into the problems that actually show up in interviews.

1. **Fundamentals** -- subarray vs. subsequence vs. subset
2. **Prefix Sum** -- precompute so range queries become $O(1)$
3. **Prefix Sum + HashMap** -- turn "does a range sum to K" into a lookup
4. **Prefix / Suffix Techniques** -- when an index needs information from both sides
5. **Difference Array** -- the mirror image of prefix sum, for range *updates*
6. **Kadane's Algorithm** -- the best contiguous segment, and its variants
7. **Intervals** -- sort first, then a single greedy pass
8. **Cyclic Sort / Index Placement** -- when values *are* indices
9. **Matrix** -- traversal, transpose, rotation, spiral, in-place tricks
10. **In-Place Array Techniques** -- read/write pointers as workspace
11. **Hashing Fundamentals** -- HashSet vs. HashMap, and the seven big hash shapes
12. **Hard / Combined Problems** -- everything above, mixed together
13. **Decision Guide** -- the recognition cheat sheet and interview Q&A

The meta-skill this whole topic is training is one question, asked at every single index while scanning:

What information from the past would let me solve the current position in $O(1)$?

Almost every pattern below is just a different answer to that one question.

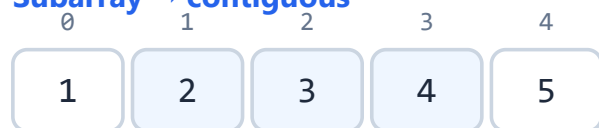
Fundamentals: Subarray vs. Subsequence vs. Subset

These three words get used loosely in casual conversation but mean very different things in interview problems, and mixing them up is one of the fastest ways to solve the wrong problem.

Concept	Meaning	Example (from [1,2,3,4,5])
Subarray	Contiguous elements	[2,3,4]
Subsequence	Order preserved, gaps allowed	[1,3,5]
Subset	Any selection, order irrelevant	{5,1}

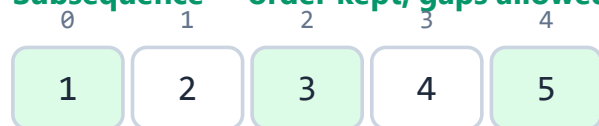
Base array: [1, 2, 3, 4, 5]

Subarray → contiguous



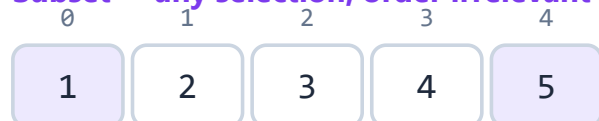
[2, 3, 4] -- indices 1,2,3, no gaps

Subsequence → order kept, gaps allowed



[1, 3, 5] -- indices 0,2,4, gaps allowed

Subset → any selection, order irrelevant



The one-line version to keep in your head while reading a problem statement:

- **Subarray** → contiguous
- **Subsequence** → order matters, gaps allowed
- **Subset** → selection, order generally irrelevant

Almost every pattern here is about **subarrays**, because contiguity lets you maintain a running summary as you scan. Subsequence and subset problems usually need DP or backtracking instead.

Part 1 -- Prefix Sum

The underlying problem

Suppose you're given `nums = [3, 2, 5, 1, 4]` and repeatedly asked for range sums:

```
sum(1, 3)
sum(0, 4)
sum(2, 4)
```

The naive approach loops through every element of every requested range. If there are `m` queries over an array of length `n`, that's $O(n * m)$ -- expensive if `m` is large.

The key question: instead of calculating every range from scratch, what information could we precompute so that the sum of *any* range can be obtained immediately?

Building the prefix array

Store cumulative sums:

index:	0	1	2	3	4
nums:	3	2	5	1	4
prefix:	3	5	10	11	15

So `prefix[i] = sum(nums[0 ... i])`. To get `sum(l, r)`, take the total up to `r` and remove everything before `l`:

```
sum(l, r) = prefix[r] - prefix[l - 1]
```

A cleaner implementation zero-pads the prefix array so there's no special case when `l == 0`:

```
prefix = [0, 3, 5, 10, 11, 15]
           before array
sum(l, r) = prefix[r + 1] - prefix[l]
```

nums	3	2	5	1	4
	0	1	2	3	4
prefix	3	5	10	11	15

$\text{prefix}[i] = \text{nums}[0] + \dots + \text{nums}[i]$
 zero-padded: $\text{prefix} = [0, 3, 5, 10, 11, 15]$

For $\text{sum}(1, 3)$: $\text{prefix}[4] - \text{prefix}[1] = 11 - 3 = 8$, which matches $\text{nums}[1] + \text{nums}[2] + \text{nums}[3] = 2 + 5 + 1 = 8$.

prefix (padded)	0	3	5	10	11	15
		↑ l=1			↑ r+1=4	

$\text{sum}(1, 3) = \text{prefix}[4] - \text{prefix}[1] = 11 - 3 = 8$
 matches: $\text{nums}[1] + \text{nums}[2] + \text{nums}[3] = 2 + 5 + 1 = 8$

Code

```
def build_prefix(nums):
    prefix = [0]
    for num in nums:
        prefix.append(prefix[-1] + num)
    return prefix
```

```
class NumArray:
    """LC 303 - Range Sum Query - Immutable."""

    def __init__(self, nums):
        self.prefix = build_prefix(nums)

    def sumRange(self, left, right):
        return self.prefix[right + 1] - self.prefix[left]
```

Complexity: building the prefix array is $O(n)$ time and $O(n)$ space; each range query afterward is $O(1)$.

Prefix Sum on a Matrix

The same idea extends to two dimensions. Instead of storing the sum of everything before one index, store the sum of the rectangle from $(0,0)$ to (r,c) . Any rectangular region can then be obtained through inclusion/exclusion:

```
desired rectangle
= large rectangle
- unwanted top
- unwanted left
+ overlapping corner    (subtracted twice, added back once)
```

```
def matrix_prefix_sum(matrix):
    """2D prefix sum: P[r][c] = sum of rectangle (0,0)..(r-1,c-1)."""
    if not matrix or not matrix[0]:
        return [[0]]
    rows, cols = len(matrix), len(matrix[0])
    P = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            P[r + 1][c + 1] = (
                matrix[r][c] + P[r][c + 1] + P[r + 1][c] - P[r][c]
            )
    return P
```

```
def matrix_region_sum(P, r1, c1, r2, c2):
    """Inclusive rectangle sum using a table from matrix_prefix_sum."""
    return (
        P[r2 + 1][c2 + 1] - P[r1][c2 + 1] - P[r2 + 1][c1] + P[r1][c1]
    )
```

Recognition

Ask: "Am I repeatedly asking about sums over ranges?" Typical signals: **range sum**, **sum between l and r**, **sum of every subarray**, **many range queries**, **subarray sum**.

But there's an important extension. If the problem asks "how many subarrays have sum K?", ordinary prefix sum alone isn't enough -- that's Part 2.

Part 2 -- Prefix Sum + HashMap

This is one of the most important array + hashing combinations, and it recurs in a surprising number of disguises.

The reformulation

Given `nums = [1, 2, 3]` and `k = 3`, we want to find subarrays whose sum is `k`. The naive approach tries every starting index and extends every possible ending index, calculating the sum each time -- $O(n^2)$.

The key is to think in prefix sums. For a subarray `(i+1 ... j)`:

```
subarray_sum = prefix[j] - prefix[i]
```

We want `prefix[j] - prefix[i] = k`. Rearranged:

```
prefix[i] = prefix[j] - k
```

This changes the problem completely. While scanning, if the current running prefix is `prefix[j]`, we only need to ask: **"Have I previously seen `prefix[j] - k`?"** A HashMap answers that in $O(1)$ average time.

`nums = [1, 2, 3], k = 3`

num	prefix	needed=prefix-k	seen before?	map after
1	1	-2	no	{0: 1, 1: 1}
2	3	0	yes -> +1	{0: 1, 1: 1, 3: 1}
3	6	3	yes -> +1	{0: 1, 1: 1, 3: 1, 6: 1}

answer = 2 (subarrays [1,2] and [3])

why {0:1} at the start: a prefix of 0 existed before index 0,
so subarrays that start at index 0 can still be detected.

Why {0: 1} at the start?

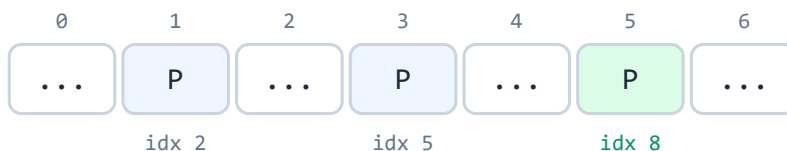
This initialization is extremely important. It means: *before processing any elements, we've seen a prefix sum of zero once*. That lets us detect subarrays that start at index 0. Without it, a case like `nums = [1, 2]`, `K = 3` would be missed -- when the running prefix reaches 3, we need to know that a prefix sum of 0 existed *before* the array began.

Counting vs. index tracking

This distinction matters a lot in prefix/hashmap problems, and mixing the two up is a common source of subtle bugs.

- **If you want to *count* subarrays:** store `prefix_sum -> frequency`. The same prefix sum can occur multiple times, and each earlier occurrence forms a different valid subarray.
- **If you want the *longest* subarray:** store `prefix_sum -> earliest index`. If the same prefix appears at indices 2, 5, and 8, using index 2 gives the longest possible subarray ending at the current position -- so never overwrite an existing entry.

Same prefix value seen at indices 2, 5, 8



Counting subarrays -> store prefix -> frequency (every occurrence counts)

Longest subarray -> store prefix -> earliest index only (2, not 5 or 8)

earliest index maximizes `current_index - stored_index`

Template -- Count Subarrays With Sum K (LC 560)

```
def subarray_sum(nums, k):  
    """LC 560 - count subarrays with sum == k."""  
    prefix_count = {0: 1}  
    prefix = 0  
    count = 0  
    for num in nums:  
        prefix += num  
        needed = prefix - k  
        count += prefix_count.get(needed, 0)  
        prefix_count[prefix] = prefix_count.get(prefix, 0) + 1  
    return count
```

Complexity: $O(n)$ time, $O(n)$ space.

Longest Subarray With Sum K (LC 325, premium)

Same derivation, but now we want the longest *distance* rather than a count, so we store the earliest index and never overwrite it.

```
def longest_subarray_sum_k(nums, k):
    """LC 325 – longest subarray with sum == k (store earliest index)."""
    first = {0: -1}
    prefix = 0
    longest = 0
    for i, num in enumerate(nums):
        prefix += num
        if prefix - k in first:
            longest = max(longest, i - first[prefix - k])
        if prefix not in first:
            first[prefix] = i
    return longest
```

Extension -- Subarray Sums Divisible by K (LC 974)

For a subarray sum to be divisible by k :

```
(prefix[j] - prefix[i]) % k == 0
=> prefix[j] % k == prefix[i] % k
```

So instead of storing the whole prefix sum, store its **remainder**. Two prefixes with the same remainder mark a subarray whose sum is a multiple of k .

```
def subarrays_div_by_k(nums, k):
    """LC 974 – count subarrays whose sum is divisible by k."""
    remainder_count = {0: 1}
    prefix = 0
    result = 0
    for num in nums:
        prefix += num
        remainder = prefix % k
        result += remainder_count.get(remainder, 0)
        remainder_count[remainder] = remainder_count.get(remainder, 0) + 1
    return result
```

Python's `%` already returns a non-negative result for negative operands as long as `k` is positive, so no extra normalization is needed here.

Extension -- Contiguous Array (LC 525): equal 0s and 1s

A clean transformation problem. Map `0 -> -1` and `1 -> +1`. "Equal number of 0s and 1s" then becomes "subarray sum == 0" -- the longest zero-sum subarray problem, solved with the same earliest-index technique as above.

```
def find_max_length(nums):
    """LC 525 - Contiguous Array: longest subarray with equal 0s and 1s."""
    first = {0: -1}
    prefix = 0
    longest = 0
    for i, num in enumerate(nums):
        prefix += 1 if num == 1 else -1
        if prefix in first:
            longest = max(longest, i - first[prefix])
        else:
            first[prefix] = i
    return longest
```

Extension -- equal counts of 0s, 1s, and 2s

A natural generalization: track two running differences, `count0 - count1` and `count1 - count2`. If two positions have the same pair of differences, the subarray between them has equal counts of all three values.

```
def equal_zero_one_two(nums):
    """Extension: count subarrays with equal counts of 0, 1 and 2."""
    count = [0, 0, 0]
    seen = {(0, 0): 1}
    result = 0
    for num in nums:
        count[num] += 1
        state = (count[0] - count[1], count[1] - count[2])
        result += seen.get(state, 0)
        seen[state] = seen.get(state, 0) + 1
    return result
```

The deeper pattern behind all four of these: **multiple equality conditions** → **represent state as a difference (or a tuple of differences)** → **same state seen before** → **valid subarray**

found via HashMap.

Recognition

When you see **subarray** + a **sum/target condition** + **count / existence / longest**, immediately consider Prefix Sum + HashMap. The deeper thought process:

current prefix

- > what previous prefix would make the difference equal K?
- > prefix - K
- > have I seen it?
- > HashMap

Part 3 -- Prefix / Suffix Techniques

The general problem

An element needs information about everything to its left AND everything to its right. Instead of repeatedly scanning left/right for every element, preprocess that information once.

```
      i
[ 3, 1, 4, 2, 5 ]
prefix -> | <- suffix
```

This converts repeated $O(n)$ scans (one per index, $O(n^2)$ total) into preprocessing + $O(1)$ lookup per element.

Prefix / suffix maximum (and minimum)

For `nums = [3, 1, 4, 2, 5]`:

```
prefix_max = [3, 3, 4, 4, 5]    prefix_max[i] = max(nums[0 ... i])
suffix_max = [5, 5, 5, 5, 5]    suffix_max[i] = max(nums[i ... n-1])
```

```
def prefix_suffix_max(nums):
    """prefix_max[i] = max(nums[0..i]); suffix_max[i] = max(nums[i..n-1])."""
    n = len(nums)
    prefix_max = [0] * n
    suffix_max = [0] * n
    running = float("-inf")
    for i in range(n):
        running = max(running, nums[i])
        prefix_max[i] = running
    running = float("-inf")
    for i in range(n - 1, -1, -1):
        running = max(running, nums[i])
        suffix_max[i] = running
    return prefix_max, suffix_max
```

Recognition: if a problem repeatedly asks for the best/largest/smallest value to the left or right of each position, think Prefix/Suffix preprocessing.

Product of Array Except Self (LC 238)

Given `nums = [1, 2, 3, 4]`, produce `output = [24, 12, 8, 6]` where `output[i]` is the product of every element except `nums[i]`. The obvious approach -- multiply every other element for every index -- is $O(n^2)$.

Step 1 -- decompose. For index `i`:

```
answer[i] = (product of everything LEFT of i) x (product of everything RIGHT of i)
```

Step 2 -- build left products. `left[i] = product of nums[0 ... i-1]`, so `left[0] = 1` (nothing before index 0).

Step 3 -- build right products. `right[i] = product of nums[i+1 ... n-1]`.

```
answer[i] = left[i] x right[i]
```

nums

0	1	2	3
1	2	3	4

left product

0	1	2	3
1	1	2	6

right product

0	1	2	3
24	12	4	1

answer = left x right

0	1	2	3
24	12	8	6

Step 4 -- collapse to $O(1)$ extra space. We don't need two separate arrays. Use the output array itself to hold the left products on the first pass, then multiply in the right products on a second, reverse pass.

```
def product_except_self(nums):
    """LC 238 – no division, O(1) extra space (output array excluded)."""
    n = len(nums)
    answer = [1] * n
    left = 1
    for i in range(n):
        answer[i] = left
        left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= right
        right *= nums[i]
    return answer
```

Complexity: $O(n)$ time, $O(1)$ extra space (the output array doesn't count for the usual formulation).

A correction flagged directly in the source chat. A tempting shortcut is `total_product / nums[i]`. The chat immediately flags this as unsafe: *"Division isn't robust"* -- because the moment the array contains a zero, the division breaks.

The tempting division shortcut breaks on zeroes



`total = 1x2x0x4 = 0 -> answer[i] = total / nums[i]`

`answer[2] = 0 / 0 -> ZeroDivisionError`

`answer[0] = 0 / 1 = 0 -> looks plausible but needs a special zero-count case to even work at all.`

Prefix/suffix products never divide, so zeroes need no special case.

For `nums = [1, 2, 0, 4]`: `total = 0`, so `answer[2] = 0 / 0` is a `ZeroDivisionError`, and every other `answer[i] = 0 / nums[i] = 0` is only correct *by accident* here (it needs a hand-written special case for "exactly one zero" vs. "two or more zeros" to work at all).

`product_except_self_division_buggy` in `sol.py` implements that special-cased version explicitly, purely to demonstrate how much extra handling division requires. The prefix/suffix-product approach above needs **zero** special cases for zeroes -- multiplying by zero naturally propagates the right answer through both passes.

nums	product_except_self answer
[1, 2, 0, 4]	[0, 0, 8, 0]

The important pattern

Don't memorize "Product of Array Except Self." Memorize the decomposition:

```
current element
[ everything left | i | everything right ]
    prefix info         suffix info
        combine both
```

This appears far beyond products: left maximum + right maximum, left minimum + right minimum, left count + right count -- the same shape, different combine step.

Recognition

When you see *"for every index, I need information about elements on both sides,"* think Prefix/Suffix, then ask: can I preprocess the left side? Can I preprocess the right side? Can I combine the two? Can I reuse the output array to hit $O(1)$ extra space?

Part 4 -- Difference Array

Prefix sum answers **many range queries**. The difference array is the mirror-image trick: it answers **many range updates**.

The problem

Start with `nums = [0]*10` and apply a batch of range updates:

```
add +5 to [2, 7]
add +3 to [4, 9]
```

The naive approach updates every element in each range individually. With `m` updates each potentially touching `n` elements, that's $O(n * m)$.

The key insight

We don't need to immediately update every element. For `add +5 to [2, 7]`, we can simply record: *"starting at index 2, everything increases by 5,"* and at index 8, *"stop applying that +5."*

```
diff[2] += 5
diff[8] -= 5
```

That's the entire trick. Applying both updates:

```
diff = [0, 0, 5, 0, 3, 0, 0, 0, -5, 0]
```

Reconstructing with a running sum gives the final array:

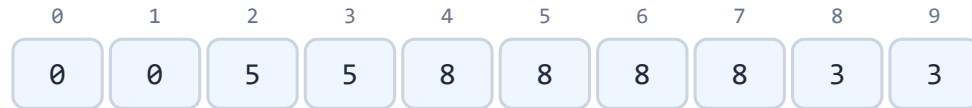
```
[0, 0, 5, 5, 8, 8, 8, 8, 3, 3]
```

add +5 to [2,7], add +3 to [4,9] (n=10)

diff marks



reconstructed



running prefix sum of the diff array = the final array

diff[l] += v ; diff[r+1] -= v -> the -v cancels the effect right after r

Why `r + 1` works

For range `[l, r]`, write `diff[l] += value` and `diff[r + 1] -= value`. Think of it as:

l	r	r+1
START +value	last	STOP

The prefix sum carries `value` forward from `l` onward; at `r + 1` the negative value cancels it exactly.

Code

```
def range_updates(n, updates):  
    """General difference-array range-update reconstruction."""  
    diff = [0] * (n + 1)  
    for l, r, value in updates:  
        diff[l] += value  
        diff[r + 1] -= value  
    result = [0] * n  
    current = 0  
    for i in range(n):  
        current += diff[i]  
        result[i] = current  
    return result
```

Applied to LC 370 (Range Addition), the update tuples are simply passed straight through:

```
def get_modified_array(length, updates):
    """LC 370 - Range Addition. updates = [[start, end, inc], ...]."""
    return range_updates(length, [(u[0], u[1], u[2]) for u in updates])
```

Complexity: building updates is $O(m)$, reconstructing is $O(n)$ -- total $O(n + m)$ and $O(n)$ space, instead of the naive $O(n * m)$.

The relationship, worth memorizing

Difference Array -> efficient range updates -> take prefix sum -> final array
 Array -> difference between consecutive elements -> Difference Array

So: **Prefix Sum** → **range queries**. **Difference Array** → **range updates**. They're inverses of the same idea, applied in opposite directions.

Recognition

If you see "add X to $[l, r]$," "increase every element in $[l, r]$," or "apply updates to ranges," ask: **can I record only where the change starts and stops, then reconstruct with a prefix sum?** That's the difference-array pattern.

Part 5 -- Kadane's Algorithm

Now the question changes shape: find the **best contiguous segment**.

Why brute force is expensive

For `nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`, there are $O(n^2)$ possible subarrays; summing each separately is $O(n^3)$ (or $O(n^2)$ with prefix sums). We can do better.

The key reasoning

Imagine processing `[4, -1, 2, 1]` with an accumulated sum of `6`. Now suppose the accumulated sum becomes strongly negative -- say `current = -10` -- and the next number is `+7`. Two choices:

```
keep the old segment:  -10 + 7 = -3
throw it away, restart: 7
```

`7 > -3`, so the negative accumulated segment is actively hurting us. **If the best subarray ending at the previous position has a negative sum, it's better to discard it and start fresh.**

The recurrence

Define `current = maximum sum of a subarray ending exactly at the current index`. At `nums[i]` there are only two options: extend the previous subarray (`current + nums[i]`), or start a new one here (`nums[i]`). Take whichever is better:

```
current = max(nums[i], current + nums[i])
```

nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]

	0	1	2	3	4	5	6	7	8
	-2	1	-3	4	-1	2	1	-5	4
cur	0	1	2	3	4	5	6	7	8
	-2	1	-2	4	3	5	6	1	5
best	0	1	2	3	4	5	6	7	8
	-2	1	1	4	4	5	6	6	6

best contiguous run [4,-1,2,1] highlighted above, sum = 6

Code

```
def max_subarray(nums):  
    """LC 53 - Maximum Subarray."""  
    current = nums[0]  
    best = nums[0]  
    for i in range(1, len(nums)):  
        current = max(nums[i], current + nums[i])  
        best = max(best, current)  
    return best
```

num	current	best
-2	-2	-2
1	1	1
-3	-2	1
4	4	4
-1	3	4
2	5	5
1	6	6
-5	1	6
4	5	6

Complexity: $O(n)$ time, $O(1)$ space.

A common-mistake warning flagged directly in the source chat. Consider `[-5, -2, -8, -1]`. The correct answer is `-1`, not `0`. That's *why* we must initialize with `current = best = nums[0]` rather than `current = best = 0` -- initializing to zero silently allows the empty subarray to "win" on an all-negative input, which isn't a valid subarray at all.

All-negative array: `[-5, -2, -8, -1]`



`current = best = 0` (WRONG init)

-> every `current = max(num, 0+num)` stays `<= 0` or resets to 0, `best` silently reports 0 -- the empty subarray, which isn't valid.

`current = best = nums[0] = -5` (correct init)

-> correctly reports -1 (the least-negative single element).

`max_subarray_buggy_empty_init` in `sol.py` implements exactly this bug for comparison.

nums	correct (init = nums[0])	buggy (init = 0)
[-5, -2, -8, -1]	-1	0

Why this is DP-like

Kadane is a very small DP: `dp[i] = max(nums[i], dp[i-1] + nums[i])`. We only ever need `dp[i-1]` to compute `dp[i]`, so the state compresses to a single variable (`dp[i-1]` -> `current`), giving `O(1)` space.

Connection to prefix sum

For a subarray `i...j`, its sum is `prefix[j] - prefix[i-1]`. So maximum subarray can theoretically be viewed as "*find the largest difference between a current prefix and an appropriate previous prefix.*" Kadane packages that reasoning into a simpler one-pass state (*best subarray ending here*) rather than tracking every previous prefix explicitly.

Prefix Sum -> useful when exact range sums matter

Kadane -> useful when the best contiguous sum matters

Minimum subarray

The same idea works in reverse -- maintain *minimum* ending here instead of maximum:

```
def min_subarray(nums):
    current = nums[0]
    best = nums[0]
    for i in range(1, len(nums)):
        current = min(nums[i], current + nums[i])
        best = min(best, current)
    return best
```

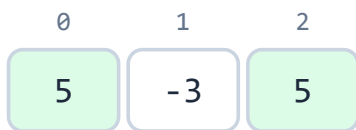
Maximum Circular Subarray (LC 918)

Now the array wraps around. For `[5, -3, 5]`, the ordinary maximum subarray is `5 + -3 + 5 = 7`, but because the array is circular we can also take `[5] + [5] = 10` by wrapping across the boundary.

A circular maximum subarray is either:

- **Case 1 -- doesn't wrap:** ordinary Kadane.
- **Case 2 -- wraps around the end:** instead of directly finding the wrapped portion, find the **minimum** subarray in the middle and remove it from the total: `circular_max = total_sum - minimum_subarray`.

`nums = [5, -3, 5] (circular)`



normal Kadane max = $5 + (-3) + 5 = 7$ (no wrap)

wrap-around max: take both edges, skip the middle

total - min_subarray = $7 - (-3) = 10$

edge case: if every number is negative, total-min = 0 (empty subarray) is invalid -- just return the ordinary Kadane result.

```
answer = max(normal_max, total_sum - minimum_subarray)
```

Important edge case: if every number is negative, `total - minimum` degenerates to the empty subarray (sum 0), which isn't valid -- return the ordinary Kadane result in that case instead.

```
def max_subarray_circular(nums):
    """LC 918 - Maximum Sum Circular Subarray."""
    total = sum(nums)
    max_current = max_best = nums[0]
    min_current = min_best = nums[0]
    for num in nums[1:]:
        max_current = max(num, max_current + num)
        max_best = max(max_best, max_current)
        min_current = min(num, min_current + num)
        min_best = min(min_best, min_current)
    if max_best < 0:
        return max_best
    return max(max_best, total - min_best)
```

Maximum Product Subarray (LC 152)

Kadane's idea gets trickier here because `negative x negative = positive` -- a very negative running product can flip to the *best* product after one more negative multiplication. So we must track **both** the maximum *and* minimum product ending at each position, swapping them whenever the current number is negative.

```
def max_product_subarray(nums):
    """LC 152 - Maximum Product Subarray (track both running max and min)."""
    current_max = nums[0]
    current_min = nums[0]
    answer = nums[0]
    for num in nums[1:]:
        if num < 0:
            current_max, current_min = current_min, current_max
        current_max = max(num, current_max * num)
        current_min = min(num, current_min * num)
        answer = max(answer, current_max)
    return answer
```

Normal Kadane only needs *best ending here*; the product version needs *best ending here* **and** *worst ending here*, because the worst can become the best.

Recognition

Maximum sum + contiguous subarray means: extend the current segment, or restart here?

Minimum: flip `max` to `min`. Circular: compare against `total - minimum`.

Part 6 -- Intervals

An interval is simply `[start, end]`. Interval problems usually become much easier after sorting - the question to ask is: **after sorting by what property does the problem become easy?**

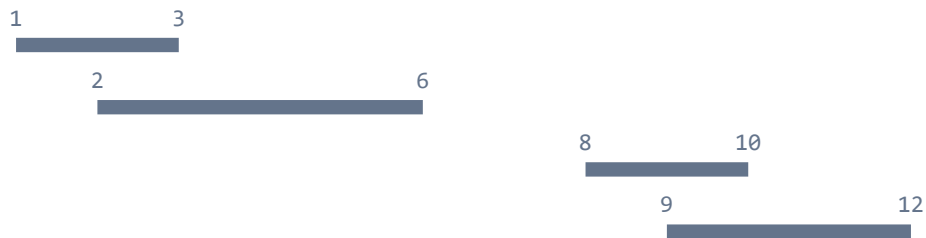
Detecting overlap

For `A = [1, 5]` and `B = [3, 7]`: they overlap because `B.start <= A.end` (`3 <= 5`). For `A = [1, 3]` and `B = [5, 7]`: no overlap, since `5 > 3`. After sorting by start, intervals can be processed left-to-right, comparing only against the most recently merged interval.

Merge Intervals (LC 56)

Given `[[1,3], [2,6], [8,10], [9,12]]`, sort by start, then walk through: `[1,3]` and `[2,6]` overlap (`2 <= 3`), merge into `[1,6]`; `[8,10]` doesn't overlap `[1,6]` (`8 > 6`), start fresh; `[8,10]` and `[9,12]` overlap (`9 <= 10`), merge into `[8,12]`.

sorted by start: `[1,3]` `[2,6]` `[8,10]` `[9,12]`



merged result:



```
def merge(intervals):
    """LC 56 – Merge Intervals."""
    if not intervals:
        return []
    intervals = sorted(intervals, key=lambda x: x[0])
    result = [list(intervals[0])]
    for start, end in intervals[1:]:
        last_end = result[-1][1]
        if start <= last_end:
            result[-1][1] = max(last_end, end)
        else:
            result.append([start, end])
    return result
```

Invariant: at every point, `result` contains intervals that are already fully merged, and only `result[-1]` can possibly overlap with the interval currently being processed -- that's the entire reason sorting by start makes this easy.

Complexity: $O(n \log n)$ for sorting, $O(n)$ for the scan.

Insert Interval (LC 57)

Now intervals are already sorted and non-overlapping, and one new interval must be inserted. Think in three phases: intervals **completely before** the new interval (no overlap possible, append directly), **overlapping** intervals (expand the new interval's bounds to absorb them), then intervals **completely after** (once `current.start > new.end`, the new interval is finished; append the rest as-is).

```
def insert_interval(intervals, new_interval):
    """LC 57 – Insert Interval."""
    result = []
    i = 0
    n = len(intervals)
    new_interval = list(new_interval)
    while i < n and intervals[i][1] < new_interval[0]:
        result.append(intervals[i])
        i += 1
    while i < n and intervals[i][0] <= new_interval[1]:
        new_interval[0] = min(new_interval[0], intervals[i][0])
        new_interval[1] = max(new_interval[1], intervals[i][1])
        i += 1
    result.append(new_interval)
    while i < n:
        result.append(intervals[i])
        i += 1
    return result
```

This is a good example of interval reasoning *without* sorting, since the input is already sorted.

Non-overlapping Intervals (LC 435)

Now the goal changes: remove the minimum number of intervals so the rest don't overlap. This is greedy. When two intervals overlap, which do we keep? Consider `A = [1,4]` , `B = [2,3]` - keeping `[1,4]` blocks everything until 4; keeping `[2,3]` frees the timeline after 3. So: **keep the interval that finishes earliest**, because it leaves the largest possible remaining region for future intervals. Sort by end.

```
def erase_overlap_intervals(intervals):
    """LC 435 – Non-overlapping Intervals (min removals). Sort by end."""
    if not intervals:
        return 0
    intervals = sorted(intervals, key=lambda x: x[1])
    removed = 0
    end = float("-inf")
    for start, finish in intervals:
        if start < end:
            removed += 1
        else:
            end = finish
    return removed
```

Complexity: $O(n \log n)$ from sorting.

Note: LC 435's real input guarantees `start < end` for every interval (no zero-length intervals) -- the greedy above assumes that, since a tie-break on `start` after sorting by `end` only matters for degenerate point intervals that can't legally appear here.

Meeting Rooms (LC 252) and Meeting Rooms II (LC 253)

Meeting Rooms asks whether one person can attend every meeting -- just detect any overlap after sorting by start.

```
def can_attend_meetings(intervals):  
    """LC 252 - Meeting Rooms."""  
    intervals = sorted(intervals, key=lambda x: x[0])  
    for i in range(1, len(intervals)):  
        if intervals[i][0] < intervals[i - 1][1]:  
            return False  
    return True
```

Meeting Rooms II asks a harder question: *how many rooms are required to hold all meetings simultaneously?* This is the maximum number of intervals active at the same time. A clean way to answer it: sort the starts and ends into two separate sorted lists, then walk them chronologically. If the next meeting starts before the earliest active meeting ends, we need another room; otherwise, a room has just freed up.

`meetings = [[0, 30], [5, 10], [15, 20]]`

	0	1	2
starts	0	5	15
ends	10	20	30

walk both sorted lists together: if next start < earliest unfinished end, open a new room; otherwise a room just freed up. max simultaneous = 2 rooms.

```
def min_meeting_rooms(intervals):
    """LC 253 - Meeting Rooms II."""
    if not intervals:
        return 0
    starts = sorted(start for start, end in intervals)
    ends = sorted(end for start, end in intervals)
    start_ptr = end_ptr = 0
    rooms = max_rooms = 0
    while start_ptr < len(intervals):
        if starts[start_ptr] < ends[end_ptr]:
            rooms += 1
            max_rooms = max(max_rooms, rooms)
            start_ptr += 1
        else:
            rooms -= 1
            end_ptr += 1
    return max_rooms
```

Recognition

When you see `[start, end]`, ask "*is this about overlap?*" Then usually: **sort** (by start or end depending on the problem) → **greedy scan**.

Merge overlapping ranges	-> sort by start
Insert a range	-> exploit sorted order
Remove minimum overlapping	-> greedy by earliest ending (sort by end)
Meeting conflict	-> detect overlap
Minimum rooms	-> track simultaneous intervals

Deeper lesson: intervals are about putting events into an order where local decisions become sufficient.

Part 7 -- Cyclic Sort / Index Placement

A very different array-specific trick. The key signal: **the values themselves give us an index.**

The idea

For `nums = [3, 1, 2, 5, 4]` where valid values are `1...n`, value `1` belongs at index `0`, value `2` belongs at index `1`, and so on -- the array can act as its own storage structure.

The fundamental operation

For `nums = [3, 1, 2]`: at index `0`, `nums[0] = 3`, which belongs at index `2`. Swap: `[2, 1, 3]`. Now index `0` holds `2`, which belongs at index `1`. Swap: `[1, 2, 3]`. Done.

start



swap `nums[0] <-> nums[2]` (3 belongs at idx 2)



swap `nums[0] <-> nums[1]` (2 belongs at idx 1) -- done



value `v` belongs at index `v-1`; don't advance `i` after a swap -- the value that just arrived may also be misplaced.

```
def cyclic_sort(nums):
    """values in 1..n placed at nums[value-1]."""
    nums = list(nums)
    i = 0
    while i < len(nums):
        correct = nums[i] - 1
        if nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    return nums
```

The important detail: don't increment `i` after a swap. The value that just arrived at `i` may also be misplaced and needs to be checked again immediately.

Missing Number (LC 268)

Values are `0..n` (one extra value compared to plain cyclic sort, since the range includes `n` itself), and exactly one is missing.

```
def missing_number(nums):
    """LC 268 - values 0..n, one missing."""
    nums = list(nums)
    i = 0
    while i < len(nums):
        correct = nums[i]
        if correct < len(nums) and nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    for i in range(len(nums)):
        if nums[i] != i:
            return i
    return len(nums)
```

Find All Numbers Disappeared in an Array (LC 448)

Values are `1..n`; after cyclic placement, any index whose value isn't correct identifies a missing number.

```
def find_disappeared_numbers(nums):
    """LC 448 – values 1..n, find every missing value."""
    nums = list(nums)
    i = 0
    while i < len(nums):
        correct = nums[i] - 1
        if nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    result = []
    for i, num in enumerate(nums):
        if num != i + 1:
            result.append(i + 1)
    return result
```

Find the Duplicate Number (LC 287)

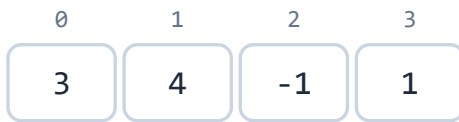
Same underlying idea -- if a value's target index is already occupied by that same value, we've found a duplicate. LC 287 specifically forbids mutating the input array, though, so the implementation below uses the closely-related **Floyd's cycle detection** trick instead: treating `nums` as a function `i -> nums[i]` turns a duplicate value into a cycle, and the cycle's entry point is exactly the duplicate.

```
def find_duplicate(nums):
    """LC 287 – Find the Duplicate Number (cyclic-sort-flavored, non-mutating
    Floyd's cycle detection variant, since LC 287 forbids modifying nums)."""
    slow = fast = nums[0]
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break
    slow2 = nums[0]
    while slow2 != slow:
        slow2 = nums[slow2]
        slow = nums[slow]
    return slow
```

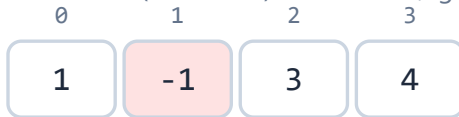
First Missing Positive (LC 41) -- the flagship hard problem

The answer must be one of `1 ... n+1`, so any value outside `[1, n]` is irrelevant and can be ignored during placement. Use the array itself as the placement structure exactly as before.

nums = [3, 4, -1, 1]



place each value v ($1 \leq v \leq n$) at index $v-1$; ignore out-of-range values



first index i where $\text{nums}[i] \neq i+1$ is index 1 $\rightarrow$ answer = 2

```
def first_missing_positive(nums):  
    """LC 41 – First Missing Positive."""  
    nums = list(nums)  
    n = len(nums)  
    i = 0  
    while i < n:  
        correct = nums[i] - 1  
        if 1 <= nums[i] <= n and nums[i] != nums[correct]:  
            nums[i], nums[correct] = nums[correct], nums[i]  
        else:  
            i += 1  
    for i in range(n):  
        if nums[i] != i + 1:  
            return i + 1  
    return n + 1
```

nums	answer
[3, 4, -1, 1]	2

Complexity: $O(n)$ time, $O(1)$ space -- despite the nested-looking `while` inside a `for`-style scan, every element is swapped into its correct position at most once, so the total work across the whole run is still linear.

Why this pattern is special

Normally, detecting "has this value been seen" costs $O(n)$ extra space via a HashSet. But if the problem *guarantees* values lie in a known range that maps naturally to indices, the array itself can serve as that storage -- trading $O(n)$ extra space for $O(1)$.

HashSet approach $\rightarrow O(n)$ extra space
Index-placement approach $\rightarrow O(1)$ extra space

When NOT to use it

Don't force cyclic sort onto arbitrary arrays. It requires a strong structural property: values in $1 \dots n$, values in $0 \dots n$, or values that otherwise correspond naturally to indices. For `[-100, 57, 923, 18]` there's no useful value $\rightarrow$ index relationship at all.

Recognition

When you see "*numbers from 1 to n*," "*missing number*," "*duplicate number*," "*each number should appear once*," "*in-place*," or " *$O(1)$ extra space*," ask: **can the value itself tell me where this element belongs?** If yes -- Cyclic Sort / Index Placement.

Part 8 -- Matrix / 2D Array Patterns

A matrix is simply an array where each element is itself an array. This section stays focused on array manipulation, not BFS/DFS (that distinction matters -- see the end of this part).

Traversal and boundaries

```
for row in matrix:
    for value in row:
        ...           # order: row by row, left to right

rows = len(matrix)
cols = len(matrix[0])
for r in range(rows):
    for c in range(cols):
        matrix[r][c]  # index-based, when coordinates matter
```

A huge fraction of matrix bugs are simply boundary mistakes. For an `m x n` matrix, valid coordinates satisfy `0 <= r < m` and `0 <= c < n`; the last row is `rows - 1`, the last column is `cols - 1`.

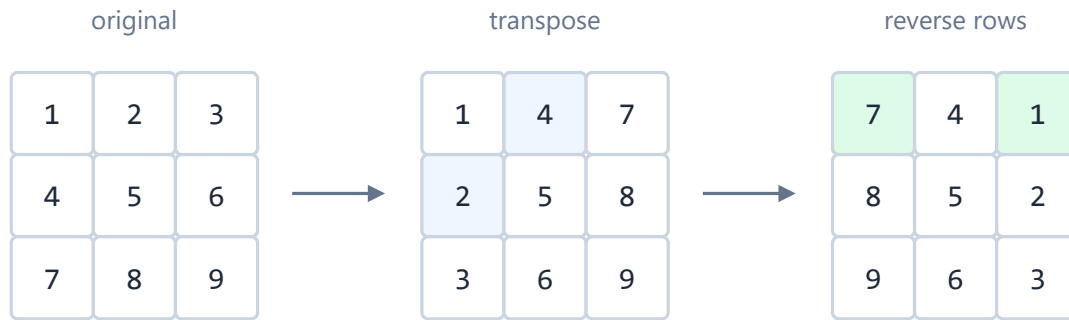
Transpose

Transpose means rows become columns and columns become rows: `transposed[c][r] = matrix[r][c]`. For a **square** matrix this can be done in place by only swapping the upper triangle (`j` starting at `i + 1`, never `i` itself, so each pair is only swapped once).

```
def transpose(matrix):
    n = len(matrix)
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    return matrix
```

Rotate Image 90 deg Clockwise (LC 48)

A very useful decomposition: **rotate = transpose, then reverse every row.**



90 deg clockwise = transpose (swap across diagonal), then reverse each row

```
def rotate(matrix):
    """LC 48 – Rotate Image 90 deg clockwise, in place."""
    n = len(matrix)
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    for row in matrix:
        row.reverse()
    return matrix
```

rotated matrix

```
[[7, 4, 1], [8, 5, 2], [9, 6, 3]]
```

Complexity: $O(n^2)$ time, $O(1)$ extra space. For 90 deg counterclockwise: transpose, then reverse **columns** instead of rows.

Spiral Matrix (LC 54)

Maintain four shrinking boundaries -- `top`, `bottom`, `left`, `right` -- and process one side at a time: top row left-to-right, right column top-to-bottom, bottom row right-to-left, left column bottom-to-top. After each side, shrink that boundary inward.

1	2	3
4	5	6
7	8	9

visit order (small numbers): top row, right col, bottom row, left col,
then shrink top/bottom/left/right and repeat until the boundaries cross

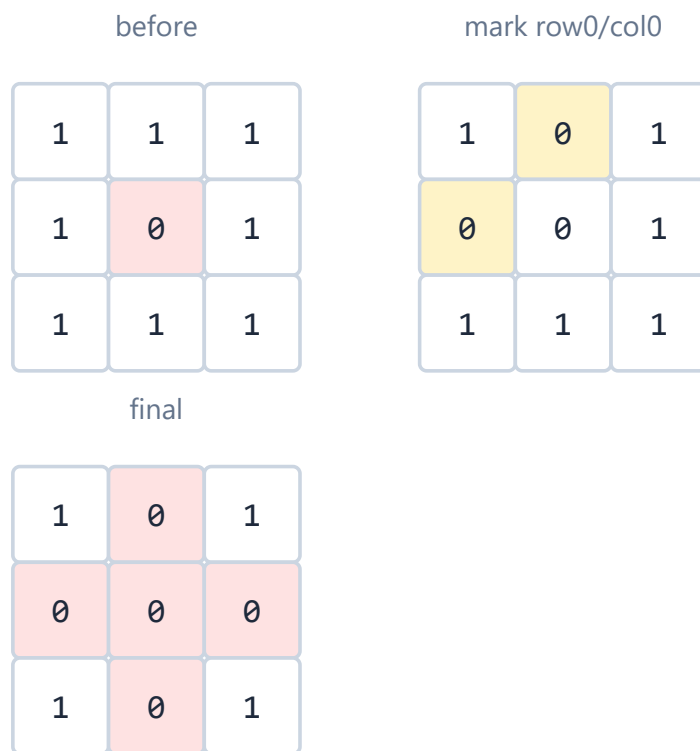
```
def spiral_order(matrix):
    """LC 54 – Spiral Matrix."""
    if not matrix or not matrix[0]:
        return []
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    result = []
    while top <= bottom and left <= right:
        for c in range(left, right + 1):
            result.append(matrix[top][c])
        top += 1
        for r in range(top, bottom + 1):
            result.append(matrix[r][right])
        right -= 1
        if top <= bottom:
            for c in range(right, left - 1, -1):
                result.append(matrix[bottom][c])
            bottom -= 1
        if left <= right:
            for r in range(bottom, top - 1, -1):
                result.append(matrix[r][left])
            left += 1
    return result
```

matrix	spiral order
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]	[1, 2, 3, 6, 9, 8, 7, 4, 5]

The `if top <= bottom / if left <= right` guards before the third and fourth sides matter specifically for rectangular matrices, so a collapsed single row or column doesn't get double-visited.

Set Matrix Zeroes (LC 73)

If any cell is `0`, its entire row and column must become `0`. The naive approach needs $O(m+n)$ extra space to remember which rows/columns to zero. The $O(1)$ -space trick: use **row 0 and column 0 themselves** as the marker storage, with two separate boolean flags remembering whether row 0 / column 0 *originally* contained a zero (since we're about to overwrite them as markers).



use row 0 and col 0 themselves as the "zero this row/col" markers -> $O(1)$ space

(two flags remember whether row0/col0 had a zero, applied last)

```

def set_zeroes(matrix):
    """LC 73 – Set Matrix Zeroes, O(1) extra space using row 0 / col 0 as
    markers, with two booleans for whether row 0 / col 0 themselves need
    zeroing."""
    rows, cols = len(matrix), len(matrix[0])
    first_row_has_zero = any(matrix[0][c] == 0 for c in range(cols))
    first_col_has_zero = any(matrix[r][0] == 0 for r in range(rows))

    for r in range(1, rows):
        for c in range(1, cols):
            if matrix[r][c] == 0:
                matrix[r][0] = 0
                matrix[0][c] = 0

    for r in range(1, rows):
        for c in range(1, cols):
            if matrix[r][0] == 0 or matrix[0][c] == 0:
                matrix[r][c] = 0

    if first_row_has_zero:
        for c in range(cols):
            matrix[0][c] = 0
    if first_col_has_zero:
        for r in range(rows):
            matrix[r][0] = 0

    return matrix

```

The order matters: mark first (scanning rows/cols 1..n using row0/col0 as storage), then apply the marks to the interior, and only *then* apply the two saved flags to row 0 and column 0 themselves -- applying them earlier would corrupt the markers before they're fully used.

Matrix as a grid

Sometimes a matrix represents connectivity rather than numbers (e.g. 0 = empty, 1 = occupied), where neighboring cells represent edges in a graph -- that's when BFS/DFS become relevant instead. Keep the distinction: **matrix manipulation** is an array problem; **matrix connectivity / regions / paths** is often a grid + BFS/DFS problem, a different toolbox entirely.

Recognition

For a 2D array ask: plain traversal? transpose/reverse? rotation? four shrinking boundaries? or are neighboring cells connected (BFS/DFS)? The main skill is **boundary management**.

Part 9 -- In-Place Array Techniques

The central question: **can I use the array itself as workspace instead of allocating another array?** Most of these are variations of the read/write (slow/fast) pointer pattern.

Remove Element (LC 27)

Given `nums = [3, 2, 2, 3]`, `val = 3`, we want the remaining elements `[2, 2]` without physically shrinking the list. Maintain a **write pointer** (`slow`) representing "*where should the next valid element go?*" while a **read pointer** (`fast`) scans every element once.

```
def remove_element(nums, val):
    """LC 27 - returns k; nums[:k] holds the answer."""
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != val:
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

After execution, `nums[:slow]` holds the answer.

Remove Duplicates from Sorted Array (LC 26)

Given `[1,1,2,2,3]`, want `[1,2,3]`. Because the array is sorted, duplicates are always adjacent, so `slow` tracks the position of the last unique element written so far.

```
def remove_duplicates(nums):
    """LC 26 - sorted array, returns k; nums[:k] holds the answer."""
    if not nums:
        return 0
    slow = 1
    for fast in range(1, len(nums)):
        if nums[fast] != nums[fast - 1]:
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

Move Zeroes

Given `[0,1,0,3,12]` , produce `[1,3,12,0,0]` -- move useful (non-zero) elements forward while preserving their relative order, then fill the remaining tail with zeroes.

Move Zeroes: `nums = [0, 1, 0, 3, 12]`



`slow` = next write position (only advances for non-zero values)

`fast` = scanner (reads every value once)



```
def move_zeroes(nums):  
    """in-place, preserves relative order of non-zero elements."""  
    slow = 0  
    for num in nums:  
        if num != 0:  
            nums[slow] = num  
            slow += 1  
    while slow < len(nums):  
        nums[slow] = 0  
        slow += 1  
    return nums
```

Partitioning

Partitioning separates an array by a condition, such as `negative | non-negative` Or `< pivot | >= pivot` . A common pattern: `left` scans for an element that belongs on the right, `right` scans for an element that belongs on the left, then swap -- this is exactly the partition step of QuickSort.

```
def partition_by_pivot(nums, pivot):
    """rearrange in-place so values < pivot come before values >= pivot."""
    left, right = 0, len(nums) - 1
    while left <= right:
        if nums[left] < pivot:
            left += 1
        elif nums[right] >= pivot:
            right -= 1
        else:
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1
    return nums
```

The recognition question: *if exact ordering doesn't matter and I only need elements separated into categories, can I rearrange them in place?*

Rotate Array (LC 189)

Given `nums` and `k`, rotate right by `k` positions in place. The clean `O(1)` -extra-space trick is the **reversal algorithm**: reverse the whole array, then reverse each of the two resulting segments independently.

```
def rotate_array(nums, k):
    """LC 189 – Rotate Array right by k, in place via the reversal trick:
    reverse the whole array, then reverse each of the two segments."""
    n = len(nums)
    k %= n

    def reverse(lo, hi):
        while lo < hi:
            nums[lo], nums[hi] = nums[hi], nums[lo]
            lo += 1
            hi -= 1

    reverse(0, n - 1)
    reverse(0, k - 1)
    reverse(k, n - 1)
    return nums
```

For `[1,2,3,4,5,6,7]`, `k=3`: reverse everything → `[7,6,5,4,3,2,1]`, reverse the first `k=3` → `[5,6,7,4,3,2,1]`, reverse the rest → `[5,6,7,1,2,3,4]` -- the correct right-rotation.

In-place + index placement

Cyclic sort (Part 7) is another form of using the array as workspace: `value -> target index`. So in-place techniques aren't one single algorithm -- they're a broader idea:

Array

- > can I rearrange existing elements?
- > can I avoid allocating another data structure?

Recognition

When a problem says **modify the array**, **$O(1)$ extra space**, **remove elements**, **move elements**, **partition**, **rearrange**, or **preserve some order**, ask: can I use two pointers? Can I overwrite positions that are no longer needed? Can I swap elements? Can the values themselves identify their target indices? This connects directly to the Two Pointer and Index Placement patterns.

Part 10 -- Hashing Fundamentals

Now we shift from array-specific tricks to how to *think* with hash tables. The implementation is never the interesting part -- the important question is always: **what information do I need to remember while scanning?**

HashSet vs. HashMap

```
HashSet -> "Do I have this?"  
HashMap -> "What information do I have about this?"
```

HashSet -- "have I seen this?"

☐ seen = {4, 7, 2, 9}

7 in seen -> True / False (existence only)

HashMap -- "what do I know about this?"

☐ freq = {4: 2, 7: 5, 2: 1}

freq[7] -> 5 (associated information: count, index, ...)

If you only need existence, a HashMap storing extra information you never use is wasted work -- recognizing which one you actually need prevents unnecessary data structures.

1. Frequency Map

One of the most common hashing patterns: `value -> count`.

```
def frequency_map(nums):  
    freq = {}  
    for num in nums:  
        freq[num] = freq.get(num, 0) + 1  
    return freq
```

`from collections import Counter; freq = Counter(nums)` does the same thing. The exact same pattern applies to character frequency (`for ch in s: freq[ch] = freq.get(ch, 0) + 1`) -- don't think of that as a separate "string technique"; it's the identical HashMap idea applied to

characters instead of numbers. Useful for character/element frequency, anagram detection, majority/frequency conditions, and counting occurrences generally.

2. Seen-Before (existence → HashSet)

Contains Duplicate (LC 217): *"have I encountered this before?"*

```
def contains_duplicate(nums):  
    """LC 217."""  
    seen = set()  
    for num in nums:  
        if num in seen:  
            return True  
        seen.add(num)  
    return False
```

3. Index Tracking

Now we want more than existence: `value -> index`. There are two common variants -- **first index** (`if num not in index: index[num] = i`) and **latest index** (always overwrite). Which one you need depends on the objective: for the *longest* subarray satisfying some condition, you usually want the **earliest** occurrence, because `current_index - earliest_index` is maximized. Don't memorize "always store the first index" -- ask *what does the index help me optimize?*

4. Complement Lookup -- Two Sum (LC 1)

Given `nums = [2, 7, 11, 15]`, `target = 9`: at `2`, ask *"what previous value would make 9?"* → `9 - 2 = 7`. Then ask *"have I already seen 7?"*

```
current value -> what value do I need? -> target - current -> HashMap lookup
```

nums = [2, 7, 11, 15], target = 9

0	1	2	3
2	7	11	15

at 2: needed = 9 - 2 = 7 -> not seen yet -> store seen[2] = 0

at 7: needed = 9 - 7 = 2 -> seen[2] exists! -> return [0, 1]

seen (value -> index) right before checking 7

2
0

```
def two_sum(nums, target):  
    """LC 1."""  
    seen = {}  
    for i, num in enumerate(nums):  
        needed = target - num  
        if needed in seen:  
            return [seen[needed], i]  
        seen[num] = i  
    return []
```

nums	target	answer
[2, 7, 11, 15]	9	[0, 1]

The ordering matters: (1) calculate the complement, (2) look for the complement, (3) *then* store the current value -- this ordering prevents using the same element twice. $O(n)$ average time, $O(n)$ space.

The **general complement pattern** generalizes far past Two Sum -- the expression changes but the reasoning stays the same: target sum $\rightarrow$ needed = target - current ; target difference $\rightarrow$ needed = current +/- target ; target XOR $\rightarrow$ needed = current XOR target .

5. HashMap + Grouping -- Group Anagrams (LC 49)

Given ["eat", "tea", "tan", "ate", "nat", "bat"] , group anagrams together. The question isn't "how do I compare every pair of strings" -- it's "*what property makes two objects belong to the same group?*"

A **canonical key**: different objects that belong to the same category should produce the same representation. For anagrams, the sorted string works ("eat" -> "aet" , "tea" -> "aet").

words: eat, tea, tan, ate, nat, bat

"aet" -> [eat, tea, ate]

"ant" -> [tan, nat]

"abt" -> [bat]

canonical key = sorted characters (or a 26-length frequency vector)

different objects, same key -> same group -> `HashMap[key].append(obj)`

```
def group_anagrams(strs):  
    """LC 49 – canonical key = sorted string."""  
    groups = defaultdict(list)  
    for s in strs:  
        key = "".join(sorted(s))  
        groups[key].append(s)  
    return list(groups.values())
```

groups

['eat', 'tea', 'ate']

['tan', 'nat']

['bat']

A faster variant avoids sorting each string by using a 26-length character-frequency vector as the key instead:

```
def group_anagrams_signature(strs):  
    """Faster variant – canonical key = 26-length frequency signature."""  
    groups = defaultdict(list)  
    for s in strs:  
        count = [0] * 26  
        for ch in s:  
            count[ord(ch) - ord("a")] += 1  
        groups[tuple(count)].append(s)  
    return list(groups.values())
```

Object -> derive canonical representation -> `HashMap` -> same representation -> same group

Extensions: more hashing-fundamentals problems

Valid Anagram (LC 242) -- two strings are anagrams iff their character-frequency counters match:

```
def valid_anagram(s, t):
    """LC 242."""
    if len(s) != len(t):
        return False
    return Counter(s) == Counter(t)
```

Ransom Note (LC 383) -- can `ransomNote` be built from letters available in `magazine`? Every needed letter's count must be covered by what's available:

```
def ransom_note(ransom_note_str, magazine):
    """LC 383."""
    avail = Counter(magazine)
    need = Counter(ransom_note_str)
    for ch, cnt in need.items():
        if avail.get(ch, 0) < cnt:
            return False
    return True
```

First Unique Character in a String (LC 387) -- build the frequency map once, then scan left-to-right for the first character whose count is exactly 1:

```
def first_uniq_char(s):
    """LC 387 -- index of the first non-repeating character, or -1."""
    freq = Counter(s)
    for i, ch in enumerate(s):
        if freq[ch] == 1:
            return i
    return -1
```

Intersection of Two Arrays II (LC 350) -- count one array, then walk the other, taking an element (and decrementing its count) each time its count is still positive -- this correctly limits the number of times a repeated value can appear in the answer to the minimum count across both arrays:

```
def intersect(nums1, nums2):
    """LC 350 – Intersection of Two Arrays II (with duplicates, by count)."""
    counts = Counter(nums1)
    result = []
    for num in nums2:
        if counts.get(num, 0) > 0:
            result.append(num)
            counts[num] -= 1
    return result
```

Encode and Decode Strings (LC 271, premium) -- serialize a list of strings into one string and back, where the strings may contain *any* character, including delimiters you might otherwise pick. The trick is a **length-prefixed encoding**: "`<len>#<string>`" repeated for every string. Because the decoder always knows exactly how many raw characters to consume next, it never needs to search for a delimiter inside the string itself -- so no character, including `#`, needs escaping.

```
class Codec:
    """LC 271 – Encode and Decode Strings.
    length-prefixed encoding: "<len>#<string><len>#<string>..."
    Robust against any characters (including '#') inside the strings
    themselves, since the length prefix tells the decoder exactly how many
    raw characters to consume next."""

    def encode(self, strs):
        parts = []
        for s in strs:
            parts.append(f"{len(s)}#{s}")
        return "".join(parts)

    def decode(self, s):
        result = []
        i = 0
        n = len(s)
        while i < n:
            j = i
            while s[j] != "#":
                j += 1
            length = int(s[i:j])
            start = j + 1
            result.append(s[start:start + length])
            i = start + length
        return result
```

The Big Hashing Patterns

- | | |
|----------------------|-----------------------------|
| 1. Frequency | value -> count |
| 2. Existence | HashSet |
| 3. Seen-before | value -> seen |
| 4. Index tracking | value -> index |
| 5. Complement lookup | needed value -> lookup |
| 6. Grouping | canonical key -> group |
| 7. Prefix + HashMap | prefix state -> count/index |

Combine Arrays + Hashing

When you see an array problem, don't immediately think "*use a HashMap*." Instead ask: **what information would I want to remember while scanning the array?**

- | | | |
|------------------------------|-----------------------------------|-------------------------------|
| Two Sum | -> need previous value | -> HashMap: value -> index |
| Contains Duplicate | -> need to know if value appeared | -> HashSet |
| Top K Frequent
(+ Heap) | -> need frequency | -> HashMap: value -> count |
| Longest Consecutive Sequence | -> need fast existence checks | -> HashSet |
| Subarray Sum K | -> need previous prefix sums | -> Prefix Sum + HashMap |
| Product Except Self | -> need left + right information | -> Prefix/Suffix |
| Group Anagrams | -> need category representation | -> Canonical key +
HashMap |

The important skill is recognizing **what information must survive the scan** -- that's the foundation for the harder combinations in Part 11.

Part 11 -- Hard / Combined Problems

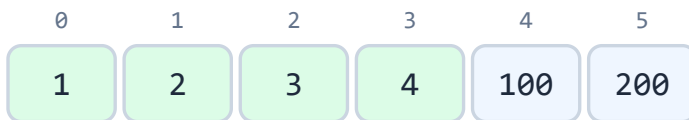
Everything from here on combines two or more of the earlier patterns. These are the problems worth being able to *derive*, not memorize.

Longest Consecutive Sequence (LC 128)

Given `[100, 4, 200, 1, 3, 2]`, find the length of the longest run of consecutive integers (order in the input doesn't matter) -- here, `[1,2,3,4]`, length 4.

The naive approach sorts first ($O(n \log n)$), but a HashSet gets this to $O(n)$ average with one key observation: **only start counting a sequence from a number that has no predecessor** (`num - 1` not in the set). Every number that *does* have a predecessor will eventually get counted as part of that predecessor's walk, so starting from every element would repeat huge amounts of work.

`nums = [100, 4, 200, 1, 3, 2] -> set = [1, 2, 3, 4, 100, 200]`



only start counting from a value with NO predecessor (v-1 not in set):

1 has no predecessor -> walk 1,2,3,4 -> length 4 (the answer)

4 DOES have a predecessor (3) -> skip starting there, avoids $O(n^2)$

```
def longest_consecutive(nums):
    """LC 128 – only start counting from a number with no predecessor."""
    num_set = set(nums)
    longest = 0
    for num in num_set:
        if num - 1 not in num_set:
            current = num
            length = 1
            while current + 1 in num_set:
                current += 1
                length += 1
            longest = max(longest, length)
    return longest
```

nums	answer
[100, 4, 200, 1, 3, 2]	4

Complexity: $O(n)$ average time and space. Even though there's a `while` loop nested inside the `for` loop, each number is only ever visited as part of a "start of sequence" walk once across the whole run, because non-start numbers immediately fail the `num - 1 not in num_set` check and never enter the inner loop.

Top K Frequent Elements (LC 347)

Two separate sub-problems: (1) find frequencies, (2) find the `k` largest frequencies. Build `value -> frequency`, then maintain a **min-heap of size k**: push each `(count, value)` pair, and pop the smallest whenever the heap exceeds size `k`. What survives is exactly the `k` largest counts.

`nums = [1, 1, 1, 2, 2, 3], k = 2`

`freq (value -> count)`

1	2	3
3	2	1

push `(count, value)` into a min-heap; when `size > k`, pop the smallest
heap keeps the `k` largest counts seen so far -> $O(n \log k)$

answer = [1, 2]

```
def top_k_frequent(nums, k):
    """LC 347 - HashMap + min-heap of size k."""
    freq = Counter(nums)
    heap = []
    for num, count in freq.items():
        heapq.heappush(heap, (count, num))
        if len(heap) > k:
            heapq.heappop(heap)
    return [num for count, num in heap]
```

nums	k	answer
[1, 1, 1, 2, 2, 3]	2	[1, 2]

Complexity: $O(n \log k)$ time, $O(n)$ space.

Majority Element (LC 169) -- Boyer-Moore Voting

Find the element appearing more than $n/2$ times, in $O(1)$ space (no HashMap at all). Maintain a `candidate` and a `count`. If `count` hits zero, pick a new candidate. Matching elements increment the count; different elements decrement it. Because the majority element occurs more than *all other elements combined*, it always survives the cancellation.

```
def majority_element(nums):
    """LC 169 -- Boyer-Moore voting, O(1) space."""
    candidate = None
    count = 0
    for num in nums:
        if count == 0:
            candidate = num
        count += 1 if num == candidate else -1
    return candidate
```

num	candidate	count
2	2	1
2	2	2
1	2	1
1	2	0
1	1	1
2	1	0
2	2	1

Minimum Size Subarray Sum (LC 209) -- a sliding-window mix-in

This combines the array concept with sliding window: because all values are positive here, growing the right pointer only ever increases the running sum, and shrinking from the left only ever decreases it -- that monotonicity is exactly what makes the two-pointer shrink-while-valid loop correct.

```
def min_subarray_len(target, nums):
    """LC 209 – shortest subarray with sum >= target (positive numbers)."""
    left = 0
    current = 0
    answer = float("inf")
    for right in range(len(nums)):
        current += nums[right]
        while current >= target:
            answer = min(answer, right - left + 1)
            current -= nums[left]
            left += 1
    return 0 if answer == float("inf") else answer
```

Distinct Elements in Every Window of Size K

Maintain a frequency map as a fixed-size window slides: increment on entry, decrement on exit, and delete the key entirely once its count hits zero so the map's size always equals the number of currently-distinct values in the window.

```
def distinct_in_windows(nums, k):
    """Number of distinct elements in every window of size k."""
    if k > len(nums):
        return []
    freq = {}
    result = []
    for i in range(k):
        freq[nums[i]] = freq.get(nums[i], 0) + 1
    result.append(len(freq))
    for right in range(k, len(nums)):
        left = right - k
        freq[nums[right]] = freq.get(nums[right], 0) + 1
        freq[nums[left]] -= 1
        if freq[nums[left]] == 0:
            del freq[nums[left]]
        result.append(len(freq))
    return result
```

Longest Subarray With At Most K Distinct

Sliding Window + HashMap: expand the right pointer, and whenever the number of distinct keys in the frequency map exceeds `k`, shrink from the left (decrementing and deleting) until it's valid again.

```
def longest_at_most_k_distinct(nums, k):
    """Longest contiguous subarray with at most k distinct values."""
    freq = {}
    left = 0
    answer = 0
    for right, num in enumerate(nums):
        freq[num] = freq.get(num, 0) + 1
        while len(freq) > k:
            freq[nums[left]] -= 1
            if freq[nums[left]] == 0:
                del freq[nums[left]]
            left += 1
        answer = max(answer, right - left + 1)
    return answer
```

Final recognition cheat sheet (this is the part worth memorizing)

CONTIGUOUS RANGE	-> Prefix Sum / Sliding Window
CONTIGUOUS + EXACT SUM	-> Prefix Sum + HashMap
COUNT subarrays with sum K	-> prefix -> frequency
LONGEST subarray with sum K	-> prefix -> earliest index
RANGE SUM QUERIES	-> Prefix Sum
RANGE UPDATES	-> Difference Array
BEST CONTIGUOUS SUM	-> Kadane
BEST CONTIGUOUS PRODUCT	-> max AND min ending here
INTERVALS	-> Sort -> Merge / Greedy
VALUES IN 1...N	-> Index Placement / Cyclic Sort
MATRIX TRANSFORMATION	-> in-place swaps / boundaries
NEED EXISTENCE	-> HashSet
NEED FREQUENCY	-> HashMap
NEED INDEX	-> HashMap[value] = index
NEED PAIR COMPLEMENT	-> HashMap[value] = index
NEED GROUPING	-> Canonical Key + HashMap
FREQUENCY + TOP K	-> HashMap + Heap
EQUAL COUNTS	-> transform into differences -> Prefix State + HashMap

The meta-pattern behind nearly all of it: **ask what information from the past would let you solve the current position in $O(1)$.**

The important hard set to be able to derive from scratch

1. Subarray Sum Equals K
2. Longest Subarray Sum K

3. Longest Consecutive Sequence
4. First Missing Positive
5. Maximum Circular Subarray
6. Maximum Product Subarray
7. Subarrays Divisible by K
8. Contiguous Array (equal 0s and 1s)
9. Equal counts of 0s, 1s, and 2s
10. Top K Frequent Elements
11. Distinct Elements in Windows
12. Longest Subarray With At Most K Distinct

These cover most of the important combinations from the source plan rather than isolated tricks.

Part 12 -- Decision Guide, Recognition Drills & Interview Q&A

Two "common mistake" warnings from the source chat

1) Kadane on all-negative arrays: current/best MUST start at `nums[0]`, not 0 -- else the empty subarray silently wins.

2) Product Except Self via `total/nums[i]`: division breaks the moment a zero appears -- prefix/suffix products need no special case at all.

Both are preserved below as dedicated worked examples with their own buggy-vs-correct functions in `sol.py`, verified by `traces.py`.

The one-question decision tree

Ask these in order when a new array problem shows up:

1. **Is it about a contiguous range or subarray?**
2. Need a fixed final answer over ranges asked repeatedly → **Prefix Sum**
3. Need to count/find subarrays hitting an exact sum/remainder/equality → **Prefix Sum + HashMap**
4. Need the *best* contiguous sum/product → **Kadane**
5. Need many range *updates* rather than queries → **Difference Array**
6. **Does an index need information from both sides?** → **Prefix/Suffix**
7. **Are the values themselves indices (1..n or 0..n)?** → **Cyclic Sort / Index Placement**
8. **Is it `[start, end]` pairs?** → **Sort, then greedy/merge (Intervals)**
9. **Is it 2D?** → **Matrix**: traversal, boundaries, transpose/rotate, spiral, or (if cells are connected) BFS/DFS instead.
10. **Do I just need to rearrange elements in place?** → **In-Place Array Techniques**
11. **None of the above, but I keep asking "have I seen this" or "what do I know about this"?** → **Hashing Fundamentals** -- pick the shape (existence / frequency / index / complement / grouping) that matches what you actually need to remember.

Recognition drills

Quick-fire: what's the first pattern you'd reach for?

Signal in the problem statement	Pattern
"sum of every subarray equal to k"	Prefix Sum + HashMap
"range sum query, many queries"	Prefix Sum
"add value to every element in [1, r], many updates"	Difference Array
"maximum sum contiguous subarray"	Kadane
"product of array except self"	Prefix/Suffix
"numbers from 1 to n, find the missing/duplicate one"	Cyclic Sort
"merge overlapping meetings"	Intervals (sort by start)
"minimum rooms needed"	Intervals (two sorted pointer sweep)
"rotate the image / matrix 90 degrees"	Transpose + reverse
"traverse in spiral order"	Four shrinking boundaries
"group words that are anagrams"	Canonical key + HashMap
"two numbers that sum to target"	Complement lookup
"longest run of consecutive integers"	HashSet, start-of-sequence check
"k most frequent elements"	HashMap + heap
"element appearing more than n/2 times, O(1) space"	Boyer-Moore voting

Interview Q&A

Q: Why does the Subarray Sum K template initialize `prefix_count = {0: 1}` instead of `{}`?

A: Without it, subarrays that start at index 0 can never be detected -- the algorithm needs to know a prefix sum of exactly 0 "existed" once, before any elements were processed.

Q: When would you store `prefix -> frequency` versus `prefix -> earliest index`? A: Frequency when *counting* subarrays (every earlier occurrence forms a different valid subarray); earliest index when finding the *longest* subarray (an earlier start maximizes `current_index - stored_index`, and later occurrences of the same prefix are strictly worse, so they should never overwrite the stored one).

Q: Why does Product of Array Except Self avoid division? A: Division breaks the moment the array contains a zero (`ZeroDivisionError` , or extra special-casing for one-vs-multiple zeroes just to patch it up). The prefix/suffix product approach needs no special case at all -- multiplying by zero naturally propagates through both passes.

Q: Why must Kadane initialize `current = best = nums[0]` , not `0` ? A: Initializing to 0 silently allows the empty subarray to "win" whenever every number is negative, which is an invalid answer for problems that require a non-empty subarray.

Q: What's the practical difference between Prefix Sum and a Difference Array? A: They answer opposite questions on the same idea. Prefix Sum precomputes a running total so **range queries** become $O(1)$. Difference Array records only where a change starts/stops so **range updates** become $O(1)$ each, reconstructed at the end with a single prefix-sum pass.

Q: Why is Boyer-Moore voting correct without ever explicitly counting anything? A: Because the majority element (by definition, appearing more than $n/2$ times) outnumbers every other element combined. Every mismatched element can cancel out at most one occurrence of the true majority element, and since there are strictly fewer "other" elements in total, the candidate can never be fully cancelled away by the time the scan finishes.

Q: When should you reach for cyclic sort instead of a HashSet? A: Only when the problem guarantees a value→index relationship (values constrained to a range like $1..n$). In that specific case, the array itself becomes free $O(1)$ -space storage instead of paying $O(n)$ for a HashSet -- but forcing this onto an array without that guarantee doesn't work.

Q: Why does Meeting Rooms II sort starts and ends into two *separate* arrays instead of sorting the interval pairs directly? A: Because the question is really "what is the maximum number of meetings active at any single instant," which only needs to know the relative order of *all* start times against *all* end times -- keeping the pairs bound together and sorting by start (like Merge Intervals) would lose exactly that cross-interval ordering information.

Q: For Set Matrix Zeroes, why record whether row 0 / column 0 had a zero *before* using them as markers, and why apply that flag last? A: Row 0 and column 0 are being repurposed as marker storage for every other row/column, so their original contents get overwritten during the marking pass. If the "should this row/col be zero" flags for row 0 and column 0 themselves were applied before the interior is processed, the scan would read already-zeroed marker cells and either over-zero the matrix or lose track of which rows/columns were really marked.